

INVESTIGACIÓN: LA EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN (1981-1990)

1: El Contexto Histórico, el Hardware y la Crisis del Software (1981-1983)

Para entender por qué los lenguajes de programación cambiaron tan drásticamente en los años 80, primero debemos entender el mundo en el que nacieron. Esta década marcó la transición definitiva de las gigantescas computadoras centrales (*mainframes*) —accesibles solo para científicos y militares— hacia la explosión de la computación personal y de oficina.

A. La revolución del hardware y las interfaces

Dos hitos tecnológicos transformaron por completo las necesidades de la industria del software:

- **La PC de IBM (1981):** Este lanzamiento estandarizó y masificó la informática en el entorno empresarial y hogareño. De pronto, había millones de computadoras en el mundo esperando por programas comerciales, bases de datos y videojuegos.
- **Las Interfaces Gráficas de Usuario (GUI):** Con la llegada de sistemas como el de la Apple Macintosh en 1984, el software dejó de ser una pantalla negra con texto verde basada en comandos. Ahora el usuario interactuaba con ventanas, botones, menús desplegables y un cursor controlado por un ratón.

B. La "Crisis del Software"

Programar una interfaz gráfica o un sistema operativo masivo con los lenguajes dominantes de los años 70 (como el C original, Fortran o Cobol) se convirtió en una pesadilla de ingeniería. Estos lenguajes eran mayoritariamente **estructurados o secuenciales**: el código se ejecutaba de arriba a abajo, línea por línea.

Si un programador quería diseñar una ventana con tres botones y un menú, el código resultante era gigantesco, enredado y propenso a errores catastróficos. Modificar una sola línea para arreglar un fallo (*bug*) podía romper por completo otra parte del sistema. Las empresas gastaban más dinero y tiempo manteniendo el código viejo que creando innovaciones. La industria necesitaba con urgencia un nuevo enfoque metodológico: **la Programación Orientada a Objetos (POO)**.

2: La Revolución de los Objetos y los Nuevos Estándares (1983-1986)

La respuesta a la crisis del software fue empaquetar los datos y las funciones en bloques independientes e interactivos llamados "**Objetos**" (como piezas de Lego). Bajo este paradigma, un botón en la pantalla es un objeto con propiedades (color, tamaño) y acciones (dar clic). A partir

de 1983, nacieron los titanes que adoptaron esta filosofía y definieron el rumbo del software moderno.

A. C++ (1983) – El motor del alto rendimiento

Creado por el científico danés **Bjarne Stroustrup** en los prestigiosos laboratorios de AT&T Bell. Su motivación era simple: amaba la velocidad y el control de hardware que ofrecía el lenguaje C, pero necesitaba el orden organizativo de los objetos. Al principio lo bautizó significativamente como "*C con Clases*".

- **Innovación:** Introdujo abstracciones de alto nivel (clases, herencia, polimorfismo) sin sacrificar la velocidad ni el acceso directo a la memoria física de la máquina.
- **Impacto:** Se convirtió en el estándar absoluto para el desarrollo de sistemas operativos (como Windows), navegadores web, suites de diseño (como Adobe) y motores de videojuegos de última generación.

B. Objective-C (1983) – El ADN de Apple

Desarrollado de forma independiente por Brad Cox y Tom Love, este lenguaje tomó una ruta distinta: fusionó la base del lenguaje C con la filosofía de "paso de mensajes" de un lenguaje experimental llamado Smalltalk.

- **El impulso de Steve Jobs:** En 1985, tras ser despedido de Apple, Jobs fundó la empresa NeXT y eligió Objective-C como el corazón de su nuevo y avanzado sistema operativo (NeXTSTEP). Cuando Apple compró NeXT en los años 90, este lenguaje regresó a casa para convertirse en la base sobre la que se construyeron macOS, iOS y, eventualmente, todo el ecosistema de aplicaciones del iPhone.

C. Ada (1983) – Ingeniería para sistemas críticos

Encargado directamente por el Departamento de Defensa de los Estados Unidos y nombrado en honor a Ada Lovelace, considerada la primera programadora de la historia.

- **Innovación:** Fue diseñado con una filosofía de hiperseguridad. Su compilador era tan estricto que no permitía ejecutar el software si existía el más mínimo riesgo de un error de gestión de memoria o de concurrencia.
- **Impacto:** Se volvió obligatorio para la aviación comercial y militar, sistemas de defensa, control de misiles y gestión de tráfico aéreo, entornos donde un fallo de software costaba vidas humanas.

D. MATLAB (1984) – El estándar científico

Creado por Cleve Moler, nació originalmente para dar a sus alumnos un acceso fácil a las librerías de álgebra lineal sin tener que programar en Fortran.

- **Impacto:** Automatizó el análisis numérico, el procesamiento de señales y el cálculo matricial, transformando radicalmente la investigación científica y las ingenierías en las universidades de todo el mundo.

3: Automatización, Scripting y las Semillas del Futuro (1987-1990)

Hacia el cierre de la década, las computadoras eran sustancialmente más potentes y la memoria RAM ya no era un recurso tan escaso. La prioridad de la industria dio un giro: la **productividad del programador y la legibilidad del código** pasaron a ser más importantes que la optimización milimétrica del hardware. Nació la era del *scripting* y las herramientas de automatización.

A. Perl (1987) – El "cuchillo suizo" de Unix

Creado por el lingüista y programador Larry Wall. Con la expansión del sistema operativo Unix en los servidores del mundo, los administradores de sistemas se enfrentaban diariamente a la tediosa tarea de procesar montañas de archivos de texto y generar reportes.

- **Impacto:** Perl nació para solucionar ese problema. Aunque su sintaxis era densa y caótica (ganándose el apodo de "lenguaje de solo escritura"), era increíblemente potente. Se convirtió en la herramienta estándar para la administración de redes y, pocos años después, fue el motor oculto que dio vida a las primeras páginas web dinámicas en los inicios del internet comercial.

B. Python (Concebido en 1989) – La legibilidad como prioridad

Durante las vacaciones de Navidad de 1989, el holandés **Guido van Rossum** decidió iniciar un proyecto de desarrollo personal para mantenerse entretenido. Su meta era crear un sucesor para el lenguaje educativo ABC, corrigiendo sus fallas, pero manteniendo su mayor virtud: la extrema facilidad de lectura.

- **Innovación:** Python eliminó el uso obligatorio de llaves { } o palabras clave como BEGIN/END para estructurar el código, obligando en su lugar a usar la indentación (espacios en blanco). Si el código no estaba visualmente ordenado, simplemente no funcionaba.
- **Impacto:** Aunque su primera versión pública se lanzó en 1991, su arquitectura diseñada a finales de los 80 sembró la semilla del que hoy en día es el lenguaje más popular del planeta, dominando los campos de la Inteligencia Artificial, la Ciencia de Datos y el desarrollo académico.

C. Haskell (1990) – La pureza de la lógica matemática

En 1990, un comité internacional de científicos de la computación consolidó el diseño de Haskell, un lenguaje basado puramente en el paradigma funcional.

- **Innovación:** A diferencia de C++ o Python, en Haskell no existen variables que cambien de estado a lo largo del tiempo; todo se resuelve mediante la evaluación de funciones matemáticas puras ($f(x)$).
- **Impacto:** Abrió el camino para la investigación de vanguardia en la teoría de lenguajes de programación y demostró cómo escribir código matemáticamente demostrable y libre de efectos secundarios imprevistos.

Conclusión del tema

La década de 1981 a 1990 no fue una simple etapa de transición tecnológica; fue el periodo en el que la programación maduró para convertirse en una verdadera ingeniería de software a gran escala. Pasamos de líneas de código rígidas y puramente secuenciales a sistemas dinámicos basados en objetos, automatización y legibilidad humana.

Si hoy en día llevamos un teléfono inteligente en el bolsillo (gracias al legado de Objective-C), si disfrutamos de videojuegos con gráficos hiperrealistas (gracias a la potencia de C++), o si interactuamos diariamente con modelos de Inteligencia Artificial (gracias a los fundamentos de diseño de Python), es porque en esta brillante década de los 80 se moldearon las herramientas y las ideas que sostienen el mundo digital contemporáneo."